

Lección 24: Architecture Decision Records

Documentar el “Por Qué” de las Decisiones

Agenda

- ¿Qué es un ADR?
 - El Problema de Decisiones No Documentadas
 - Plantilla de ADR
 - Cuándo Crear ADRs
 - Ejemplos Prácticos
 - Flujo de Trabajo de ADR
 - Mejores Prácticas
-

El Problema

6 meses después del proyecto:

Nuevo Developer: "¿Por qué elegimos Zustand en vez de Redux?"

Team Lead: "Hmm, creo que fue porque... eh..."

Developer 2: "Pensé que era por el tamaño del bundle?"

Developer 3: "No, recuerdo que teníamos problemas de rendimiento con Redux..."

Developer Original: (dejó la empresa)

Resultado: Nadie sabe por qué

La decisión podría estar mal para necesidades actuales
Miedo de cambiarla

El Costo de Decisiones Olvidadas (1/2)

Sin ADRs:

1. Decisión tomada en hilo de Slack
2. Contexto perdido después de 90 días (retención de Slack)
3. Nuevo developer la cuestiona
4. El equipo debate lo mismo otra vez
5. Se llega a una conclusión diferente
6. Se reimplementa con nueva elección
7. Se descubre que el razonamiento original era correcto
8. Se revierte

Tiempo perdido: Semanas
Moral perdida: Alta

El Costo de Decisiones Olvidadas (2/2)

Con ADRs:

1. Decisión documentada en ADR
2. Contexto preservado para siempre
3. Nuevo developer lee ADR
4. Entiende el razonamiento
5. Puede cuestionar si es necesario
6. Discusión basada en hechos

Tiempo perdido: 0
Conocimiento retenido: 100%

¿Qué es un ADR?

Architecture Decision Record:

Documento corto (~1 página) que captura:

- Qué decisión se tomó
- Qué contexto llevó a ella
- Qué opciones se consideraron
- Por qué se eligió esta opción
- Cuáles son las consecuencias

Almacenado en: Repositorio Git

Formato: Markdown

Ubicación: docs/adr/

Inmutable: Nunca eliminar, solo reemplazar

Principio clave: Los documentos son inmutables. Las malas decisiones permanecen documentadas. Esto muestra la evolución del pensamiento a lo largo del tiempo.

Plantilla de ADR

ADR-XXX: [Título de la Decisión]

****Fecha**:** YYYY-MM-DD

****Estado**:** Propuesto | Aceptado | Deprecado | Reemplazado

Contexto

¿Cuál es el problema o situación que requiere una decisión?

¿Qué restricciones existen?

Opciones Consideradas

1. Opción A

2. Opción B

3. Opción C

Decisión

Elegimos [Opción B]

Justificación

Por qué esta opción sobre otras:

- Razón 1
- Razón 2

- Razón 3

Consecuencias

Positivas

- Beneficio 1
- Beneficio 2

Negativas

- Compromiso 1
- Compromiso 2

Referencias

- [Enlace a investigación]
- [Enlace a spike PR]

Ejemplo de ADR: Gestión de Estado

ADR-001: Usar Zustand para Gestión de Estado

****Fecha**:** 2024-01-10

****Estado**:** Aceptado

Contexto

La aplicación de carrito de compras necesita gestión de estado global para:

- Items del carrito
- Autenticación de usuario
- Estado de UI (modales, notificaciones)

Actual: React Context + useState causando problemas de rendimiento con re-renders excesivos en actualizaciones del carrito.

Opciones Consideradas

1. ****Redux Toolkit****

- Más popular
- Ecosistema maduro
- Tamaño de bundle grande (15KB)
- Más boilerplate

2. ****Zustand****

- Boilerplate mínimo
- TypeScript-first
- Bundle pequeño (3KB)
- Soporte de DevTools
- Equipo no familiarizado

3. ****Recoil****

- Modelo basado en átomos
- Experimental (Meta)
- Estabilidad en producción desconocida

4. ****Mantener Context API****

- Sin dependencias
- Mal rendimiento a escala
- Sin DevTools

Decisión

Usar ****Zustand****

Justificación

- ****Rendimiento****: Renderiza solo componentes cambiados (probado 10x más rápido que Context en carrito con 50+ items)
- ****Tamaño de bundle****: 3KB vs 15KB para Redux (presupuesto total de 2MB)
- ****Experiencia de desarrollador****: API simple, boilerplate mínimo
- ****TypeScript****: Soporte TS de primera clase, inferencia de tipos funciona
- ****DevTools****: Compatible con Redux DevTools
- ****Ruta de migración****: Puede migrar a Redux si es necesario (patrones similares)
- ****Curva de aprendizaje****: Equipo puede aprender en 1 día (spike completado)

Consecuencias

Positivas

- Actualizaciones de carrito más rápidas (50ms → 5ms)
- Ahorro de 12KB en bundle
- Menos boilerplate (30% menos código que Redux)
- Mejor autocompletado de TypeScript
- Testing más fácil (no necesita Provider)

Negativas

- Equipo necesita aprender nueva librería (1-2 días)
- Menos soluciones en Stack Overflow que Redux
- Puede necesitar migrar si Zustand es abandonado (improbable)

Referencias

- [Documentación de Zustand] (<https://github.com/pmndrs/zustand>)
- [Performance spike PR] (<https://github.com/company/app/pull/123>)
- [Análisis de bundle] (<https://company.atlassian.net/browse/ENG-456>)

Cuándo Crear un ADR (1/2)

Crear ADRs para:

- ✓ Elección de framework (React vs Vue)
- ✓ Gestión de estado (Redux vs Zustand)
- ✓ Estrategia de testing (Vitest vs Jest)
- ✓ Elección de base de datos (PostgreSQL vs MongoDB)
- ✓ Patrones de arquitectura (monolito vs microservicios)
- ✓ Enfoque de seguridad (JWT vs sessions)
- ✓ Plataforma de hosting (Vercel vs AWS)
- ✓ Cambios que rompen APIs públicas

Cuándo Crear un ADR (2/2)

No crear ADRs para:

- ✗ Nombrar una variable
- ✗ Agregar una función de utilidad
- ✗ Corregir un bug
- ✗ Actualizar una dependencia (a menos que sea cambio mayor que rompe)
- ✗ Refactorizar implementación interna

Regla general: Si la decisión afecta a múltiples desarrolladores o es difícil de revertir, escribe un ADR.

Ejemplo de ADR: Estrategia de Testing

ADR-002: Usar Vitest en Lugar de Jest

****Fecha**:** 2024-01-15

****Estado**:** Aceptado

Contexto

Necesidad de elegir framework de testing para nuevo proyecto.
Usando Vite para bundling. Queremos tests rápidos y buena DX.

Opciones Consideradas

1. ****Jest**** – Estándar de la industria, maduro
2. ****Vitest**** – Nativo de Vite, más rápido
3. ****Testing Library + Playwright solamente**** – Omitir tests unitarios

Decisión

Usar ****Vitest**** para tests unitarios/integración.

Justificación

- ****Velocidad****: 10x más rápido que Jest (300ms vs 3s para 50 tests)
- ****Integración con Vite****: Comparte config de Vite, sin duplicación
- ****ESM nativo****: No necesita transformación, HMR instantáneo
- ****Compatible con Jest****: Existe ruta de migración si es necesario
- ****Desarrollo activo****: Respaldado por equipo de Vite

Consecuencias

Positivas

- Ejecución rápida de tests (CI de 5min → 30sec)
- HMR para tests (editar test, re-ejecución instantánea)
- Sin duplicación de config
- Mejor modo watch

Negativas

- Librería más nueva (menos madura que Jest)
- Ecosistema más pequeño (menos plugins)
- Algunos plugins de Jest incompatibles

Referencias

- [Benchmark de Vitest](<https://vitest.dev/guide/comparisons.html>)
- [Guía de migración](<https://vitest.dev/guide/migration.html>)

Estados de ADR (1/2)

Propuesto:

Todavía en discusión
Aún no implementado
Recopilando retroalimentación

Aceptado:

Decisión tomada
Equipo está de acuerdo
Implementación en progreso o completa

Estados de ADR (2/2)

Deprecado:

Ya no se recomienda
Existe mejor opción
No usar para código nuevo
Código legacy puede aún usarlo

Reemplazado:

Reemplazado por ADR más nuevo
Enlaza al nuevo ADR
Explica qué cambió

Ejemplo de ADR: Reemplazado

ADR-003: Usar LocalStorage para Persistencia del Carrito

****Fecha**:** 2024-01-20

****Estado**:** Reemplazado por ADR-007

Decisión

Almacenar carrito anónimo en localStorage.

Reemplazado Por

[ADR-007: Estrategia Híbrida de Almacenamiento de Carrito (LocalStorage + D

Por Qué Fue Reemplazado

La decisión inicial no consideró:

- Sincronización multi-dispositivo para usuarios autenticados
- Expiración del carrito (localStorage persiste para siempre)
- Analytics en carritos abandonados (necesita datos server-side)

El nuevo enfoque híbrido resuelve estos problemas mientras mantiene los beneficios de localStorage para usuarios anónimos.

ADR-007: Estrategia Híbrida de Almacenamiento de Carrito

****Fecha**:** 2024-02-15

****Estado**:** Aceptado

****Reemplaza**:** ADR-003

Contexto

ADR-003 eligió localStorage, pero ahora necesitamos:

- Sincronización multi-dispositivo
- Analytics del carrito
- Emails de carrito abandonado

Decisión

Usar ****localStorage para anónimos****, ****database para autenticados****.

Justificación

- 90% de usuarios navegan anónimamente → localStorage = rápido
- 10% usuarios autenticados necesitan sync → database requerida
- Híbrido: lo mejor de ambos mundos

[Detalles completos...]

Flujo de Trabajo de ADR (1/3)

1. Proponer:

ADR-004: Use Playwright for E2E Testing

****Status**:** Proposed

****Date**:** 2024-03-01

[Write proposal...]

Flujo de Trabajo de ADR (2/3)

2. Discutir:

```
# Crear PR con ADR
git checkout -b adr/004-playwright
git add docs/adr/004-playwright.md
git commit -m "docs: proponer Playwright para testing E2E"
git push origin adr/004-playwright
```

```
# Discusión del PR:
# - Equipo revisa opciones
# - Preguntas respondidas
# - Decisión tomada
```

Flujo de Trabajo de ADR (3/3)

3. Aceptar:

Actualizar estado

****Estado**:** Aceptado

Hacer merge del PR

4. Implementar:

```
# El PR de implementación referencia al ADR
git commit -m "feat: agregar tests E2E con Playwright (ADR-004)"
```

ADRs en el Código (1/2)

Referenciar ADRs en comentarios del código:

```
// Usando Zustand según ADR-001: Gestión de Estado
import { create } from 'zustand'

export const useCartStore = create<CartState>(set => ({
  items: [],
  addItem: item =>
    set(state => ({
      items: [...state.items, item],
    })),
}))

// Config de Vitest según ADR-002: Estrategia de Testing
export default defineConfig({
  test: {
    globals: true,
    environment: 'jsdom',
  },
})
```

ADRs en el Código (2/2)

Enlazar ADRs en PRs:

feat: implementar persistencia del carrito

Implementa estrategia de localStorage según ADR-003.

Shopping Cart: Ejemplos de ADR (1/2)

Estructura de archivos:

```
docs/adr/
├── README.md
├── 001-zustand-state-management.md
├── 002-vitest-testing.md
├── 003-cart-persistence.md
├── 004-playwright-e2e.md
├── 005-sentry-observability.md
└── template.md
```

Shopping Cart: Ejemplos de ADR (2/2)

README.md:

Architecture Decision Records

Activos

- [ADR-001](001-zustand-state-management.md) - Zustand para Estado
- [ADR-002](002-vitest-testing.md) - Vitest para Testing
- [ADR-004](004-playwright-e2e.md) - Playwright para E2E
- [ADR-005](005-sentry-observability.md) - Sentry para Observabilidad

Reemplazados

- [ADR-003](003-cart-persistence.md) - Reemplazado por ADR-007

ADRs Asistidos por IA (1/3)

Prompt de ChatGPT para ADR:

Necesito escribir un ADR para elegir entre Redux y Zustand para gestión de estado en un carrito de compras React.

Contexto:

- Problemas de rendimiento con Context API
- Equipo familiarizado con Redux
- Presupuesto de bundle size ajustado

Genera un ADR siguiendo la plantilla estándar.

ADRs Asistidos por IA (2/3)

Prompt de Claude Code:

Crea ADR-006 para elegir TypeScript sobre JavaScript.

Contexto: Proyecto nuevo, equipo experimentado con ambos.

Decisión: TypeScript

Justificación: Type safety, mejor soporte IDE, detectar errores temprano

Consecuencias: Onboarding más largo, build step requerido

ADRs Asistidos por IA (3/3)

La IA puede:

- Generar borrador inicial (humano revisa)
- Sugerir opciones a considerar
- Identificar consecuencias
- Formatear consistentemente

La IA no puede:

- Tomar la decisión
 - Entender contexto completo
 - Reemplazar discusión del equipo
-

Mejores Prácticas (1/2)

1. Escribir ADRs temprano:

- ✗ Escribir ADR después de la implementación
 - Sesgo de confirmación
 - Alternativas faltantes
- ✓ Escribir ADR antes de la implementación
 - Pensamiento claro
 - Alineación del equipo
 - Mejor decisión

2. Mantenerlos cortos:

Objetivo: 1 página (~300–500 palabras)

Máximo: 2 páginas

Si es más largo → Dividir en múltiples ADRs

Mejores Prácticas (cont.)

3. Enfocarse en “por qué”, no “cómo”:

- ✓ "Elegimos Zustand por el tamaño de bundle y DX"
- ✗ "Así es como configurar Zustand: ..."
(Eso es documentación, no ADR)

Mejores Prácticas (2/2) (1/2)

4. Actualizar estado, nunca eliminar:

- ✗ Eliminar ADR cuando cambia la decisión
 - Historia perdida
 - No se puede aprender de errores

- ✓ Marcar como Reemplazado, enlazar a nuevo ADR
 - Historia completa
 - Evolución visible
 - Aprender del pasado

5. Enlazar a docs de soporte:

References

- [Performance benchmark](link-to-benchmark.md)
- [Spike PR](https://github.com/company/app/pull/123)
- [Team discussion](https://company.slack.com/archives/...)
- [External article](https://blog.example.com/redux-vs-zustand)

Mejores Prácticas (2/2) (2/2)

6. Revisar en PR:

Los ADRs deben pasar por revisión de PR

- Input del equipo en la decisión
- Verificar opciones faltantes
- Verificar justificación
- Discutir consecuencias

Repositorio de Plantillas

Crear plantilla reutilizable:

```
<!-- docs/adr/template.md -->
```

ADR-XXX: [Título en Modo Imperativo]

****Fecha**:** YYYY-MM-DD

****Estado**:** Propuesto

****Decisores**:** [@persona1, @persona2]

Contexto

¿Cuál es el problema que enfrentamos?
¿Qué restricciones existen?

Opciones Consideradas

1. [Opción 1]
2. [Opción 2]
3. [Opción 3]

Decisión

Elegimos [Opción X]

Justificación

- Razón 1
- Razón 2
- Razón 3

Consecuencias

Positivas

- Beneficio 1

Negativas

- Compromiso 1

Referencias

- [Enlace 1]

Copiar plantilla para nuevo ADR:

```
cp docs/adr/template.md docs/adr/008-new-decision.md
```

Ejercicio 1: Crear ADR para Decisión Arquitectónica

Prompt:

Actúa como un technical lead documentando decisión arquitectónica con ADR (

CONTEXT0: ADR = short doc (~1 page) que captura decisión arquitectónica con rationale + consequences. WHY: decisiones olvidadas → debates repetidos → t ADRs preservan contexto FOREVER (en Git). ADR structure: Context (problema) (qué elegimos), Options Considered (alternativas evaluadas), Rationale (por opción), Consequences (positive + negative outcomes). Status: Proposed (pro Accepted (implementada), Deprecated (no usar), Superseded (reemplazada por Immutable: NUNCA borrar ADRs, solo marcar superseded (historia completa vis Location: docs/adr/ en Git repo. Docs as Code: ADR vive con código, updated PR. Rule of thumb: escribir ADR si decisión afecta 2+ developers 0 es hard

TAREA: Escribe ADR-001 documentando elección de Zustand para state manageme

IMPLEMENTACIÓN:

- Archivo: docs/adr/001-state-management-library.md
- Status: Accepted
- Date: Fecha actual (YYYY-MM-DD)
- Decision: Usar Zustand (no Redux)

ADR TEMPLATE:

ADR-001: State Management Library Selection

****Date**:** YYYY-MM-DD

****Status**:** Accepted

Context

[Describe el problema: app necesita global state para cart items, team size, timeline, constraints como bundle size budget]

Options Considered

1. Redux Toolkit - [pros/cons]
2. Zustand - [pros/cons]
3. Context API - [pros/cons]

Decision

Use Zustand **for** global state management.

Rationale

- Reason 1: Bundle size (3KB vs 15KB Redux)
- Reason 2: Developer experience (minimal boilerplate)
- Reason 3: Performance (selective re-renders)

- Reason 4: TypeScript support (first-class)

Consequences

Positive

- + Benefit 1 (with specific metrics **if** possible)
- + Benefit 2

Negative

- Tradeoff 1
- Tradeoff 2

References

- [Link to benchmark **if** exists]

ELEMENTOS CRÍTICOS:

- Context: explica problema/constraints (NO solo "necesitamos state")
- Options: listar 3+ alternativas evaluadas (shows due diligence)
- Rationale: specific reasons con data (NO "es mejor")
- Consequences: honest about tradeoffs (NO solo positives)

VALIDACIÓN: ADR debe responder "¿Por qué Zustand y no Redux?" con specific

Aprende: ADRs documentan el WHY de decisiones,

preservando contexto cuando developers originales se van

Ejercicio 2: Supersede ADR (Evolución de Decisión)

Prompt:

Actúa como un tech lead documentando cambio de decisión arquitectónica con

CONTEXT0: Superseded ADR = nuevo ADR que reemplaza decisión anterior. WHY s requirements cambiaron, nueva info disponible, decisión original no escaló
Key principle: NUNCA borrar ADR antiguo, solo marcar Status: Superseded + 1 ADR. Historia completa: muestra evolución de thinking over time (team apren errores). Superseding ADR debe: 1) explicar QUÉ cambió desde decisión origi justificar por qué cambio es necesario ahora, 3) incluir migration plan si

Status progression: Proposed → Accepted → (tiempo pasa) → Superseded. Original agregar "**Superseded by ADR-XXX**" + mantener contenido original intacto. Less superseded ADRs muestran qué assumptions fallaron (previene repeat mistakes)

TAREA: Crea ADR-002 que supersede ADR-001 por cambio de requirements.

ESCENARIO:

- 2 meses después, app complexity creció significativamente
- Zustand (ADR-001) ya no es suficiente
- Necesitamos Redux Toolkit ahora
- 15+ state slices, async actions complejas, time-travel debugging

IMPLEMENTACIÓN:

- Archivo 1: Actualizar docs/adr/001-state-management-library.md
 - Cambiar Status: Accepted → Superseded by ADR-002
 - NO cambiar contenido original (immutable **history**)
- Archivo 2: Crear docs/adr/002-migrate-to-redux-toolkit.md
 - Status: Accepted
 - Supersedes: ADR-001
 - Date: Fecha 2-3 meses después de ADR-001

ADR-002 STRUCTURE:

ADR-002: Migrate to Redux Toolkit

****Date****: YYYY-MM-DD (2-3 months after ADR-001)

****Status****: Accepted

****Supersedes****: ADR-001

Context

Since ADR-001 (Zustand decision), app grew significantly:

- State slices: 3 → 15+ (cart, user, products, orders, etc.)
- Async actions: Simple → Complex (optimistic updates, retries)
- Debugging needs: Basic → Advanced (time-travel, action replay)
- Team feedback: Zustand hard to manage at this scale

Decision

Migrate from Zustand to Redux Toolkit.

Rationale

[Specific reasons why change is needed NOW vs continuing with Zustand]

Migration Plan

[Step-by-step migration with timeline]

Consequences

[Include migration cost + long-term benefits]

UPDATE ORIGINAL ADR-001:

ADR-001: State Management Library Selection

Date: 2025-01-15

Status: Superseded by ADR-002

Superseded on: 2025-03-20

[Original content unchanged]

Superseded By

[ADR-002: Migrate to Redux Toolkit](./002-migrate-to-redux-toolkit.md)

Original decision worked **for** 2 months but app outgrew Zustand's **simplicity**.

VALIDACIÓN: Historia completa debe estar visible (ADR-001 original + ADR-00

Aprende: Superseding ADRs documenta evolución sin borrar historia,

mostrando learning process del team

Puntos Clave

1. **ADRs:** Documentan decisiones arquitectónicas con contexto
2. **El Problema:** Decisiones olvidadas, contexto perdido, debates repetidos
3. **Plantilla:** Contexto, Opciones, Decisión, Justificación, Consecuencias
4. **Cuándo:** Elección de frameworks, patrones de arquitectura, decisiones difíciles de revertir
5. **Estados:** Propuesto → Aceptado → Reemplazado (nunca eliminado)
6. **Flujo de Trabajo:** Proponer → Discutir en PR → Aceptar → Implementar
7. **Mantener Corto:** 1 página, enfoque en “por qué” no “cómo”
8. **Inmutable:** Marcar como reemplazado, no eliminar (aprender de la historia)
9. **En Código:** Referenciar ADRs en comentarios y PRs

10. **Asistido por IA:** Generar borradores, pero humanos deciden